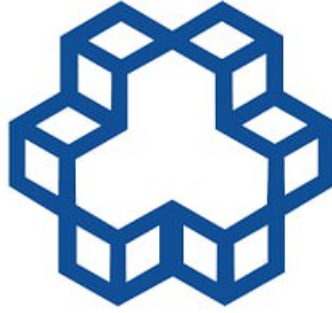




K.N. Toosi University

Aidin Sahneh

Student ID: 40120243

Amir Hosseinpour

Student ID: 40117393

Hamed Behrouzi Moghaddam

Student ID: 40215633

Course: Introduction to MachineLearning

Instructor: Dr. Nikoofard

Project Source Code: [Interactive Google Colab Notebook \(Question 5\)](#)

Contents

1	Question 1: Analysis of Forward / Backward Propagation and Dying ReLUs	2
1.1	(a) Forward pass	3
1.2	(b) Gradient with respect to $a^{[1]}$	3
1.3	(c) Gradients w.r.t. $W^{[1]}$ and x	4
1.4	(d) Dying ReLU and weight updates	5
1.5	Network Diagram	6
2	Question 2 (Problem Summary)	6
2.1	(a) Vanishing Gradients in Deep Sigmoid Networks	6
2.2	(b) ReLU, Large Learning Rates, and Batch Normalization	7
3	Question 3: Vectorized Implementation of Backpropagation and Adam	9
3.1	Task 1: Computing Gradients for Layer 1	9
3.2	Task 2: Adam Optimizer Updates for $W^{[1]}$	10
3.3	Task 3: Analytical Question on <code>keepdims</code> and Broadcasting	11
3.4	Complete Python Implementation	12
4	Question 4: Theoretical Analysis of Dropout and Scaling	12
4.1	(a) Statistical Expectation in the Training Phase	12
4.2	(b) Inference Discrepancy and Calibration Strategies	13
5	Question 5: Exoplanet Habitable Zone (Coding Challenge)	13
5.1	Phase 1: Proving the Deadlock of Linear Models	13
5.2	Phase 2: The Minimalist Architect Challenge	15
5.3	Phase 3: The Activation Trap (Vanishing Gradients)	17

1 Question 1: Analysis of Forward / Backward Propagation and Dying ReLUs

Consider a binary classification MLP with one hidden layer and one output neuron.

- Input: $x \in \mathbb{R}^2$.
- Hidden layer: 2 neurons with ReLU activation.
- Output layer: 1 neuron with sigmoid activation.
- Loss: Binary Cross-Entropy (BCE) with L2 regularization (weight decay) coefficient $\lambda = 0.1$.

We are given a single training example

$$x = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad y = 1.$$

The initial parameters are

$$W^{[1]} = \begin{bmatrix} 0.5 & 1.0 \\ 0.5 & -2.0 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix},$$

$$W^{[2]} = [1.0 \quad -1.0], \quad b^{[2]} = [0.0].$$

The network computations are:

$$z^{[1]} = W^{[1]}x + b^{[1]}, \quad a^{[1]} = \text{ReLU}(z^{[1]}),$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}, \quad \hat{y} = a^{[2]} = \sigma(z^{[2]}),$$

where $\sigma(u) = \frac{1}{1 + e^{-u}}$ is the sigmoid function, and $\text{ReLU}(u) = \max(0, u)$ is applied elementwise.

ReLU and Sigmoid Activation Functions

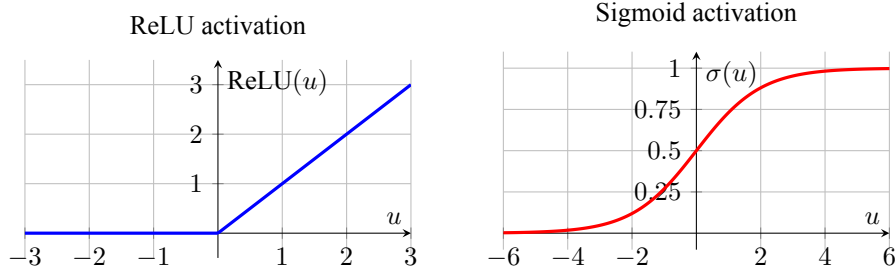


Figure 1: ReLU and sigmoid activation functions used in the hidden and output layers, respectively.

The loss for a single sample is

$$\mathcal{L} = \text{BCE}(y, \hat{y}) + \frac{\lambda}{2} (\|W^{[1]}\|_F^2 + \|W^{[2]}\|_2^2),$$

where

$$\text{BCE}(y, \hat{y}) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})).$$

Tasks

- Compute the forward-pass values $z^{[1]}, a^{[1]}, z^{[2]}, \hat{y} = a^{[2]}$ and the final loss $\mathcal{L}$.
- Compute $\frac{\partial \mathcal{L}}{\partial a^{[1]}}$.
- Using backpropagation (including ReLU and L2 regularization), compute $\frac{\partial \mathcal{L}}{\partial W^{[1]}}$ and $\frac{\partial \mathcal{L}}{\partial x}$.
- Suppose the first hidden neuron is a “dying ReLU”, i.e. its pre-activation is always non-positive and therefore its ReLU output and gradient remain zero. Does the training algorithm still update that neuron’s weights? Explain using the gradient formulas.

Solution

1.1 (a) Forward pass

First layer pre-activation.

$$W^{[1]}x = \begin{bmatrix} 0.5 & 1.0 \\ 0.5 & -2.0 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.5 \cdot 2 + 1.0 \cdot (-1) \\ 0.5 \cdot 2 + (-2.0) \cdot (-1) \end{bmatrix} = \begin{bmatrix} 0.0 \\ 3.0 \end{bmatrix}.$$

Adding the bias:

$$z^{[1]} = W^{[1]}x + b^{[1]} = \begin{bmatrix} 0.0 \\ 3.0 \end{bmatrix} + \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 3.5 \end{bmatrix}.$$

Hidden activations (ReLU).

$$a^{[1]} = \text{ReLU}(z^{[1]}) = \begin{bmatrix} \max(0, -1.5) \\ \max(0, 3.5) \end{bmatrix} = \begin{bmatrix} 0 \\ 3.5 \end{bmatrix}.$$

Output pre-activation.

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} = \begin{bmatrix} 1.0 & -1.0 \end{bmatrix} \begin{bmatrix} 0 \\ 3.5 \end{bmatrix} + 0 = 0 - 3.5 = -3.5.$$

Output activation (sigmoid).

$$\hat{y} = \sigma(z^{[2]}) = \frac{1}{1 + e^{-(-3.5)}} = \frac{1}{1 + e^{3.5}} \approx 0.0293122.$$

BCE loss (without regularization). Since $y = 1$,

$$\text{BCE}(y, \hat{y}) = -\log(\hat{y}) \approx -\log(0.0293122) \approx 3.527.$$

L2 regularization term.

$$\|W^{[1]}\|_F^2 = 0.5^2 + 1.0^2 + 0.5^2 + (-2.0)^2 = 0.25 + 1 + 0.25 + 4 = 5.5,$$

$$\|W^{[2]}\|_2^2 = 1.0^2 + (-1.0)^2 = 2.$$

Total:

$$\|W^{[1]}\|_F^2 + \|W^{[2]}\|_2^2 = 7.5.$$

Thus

$$\mathcal{L}_{\text{reg}} = \frac{\lambda}{2} \cdot 7.5 = \frac{0.1}{2} \cdot 7.5 = 0.375.$$

Total loss.

$$\mathcal{L} = \text{BCE}(y, \hat{y}) + \mathcal{L}_{\text{reg}} \approx 3.527 + 0.375 = 3.902.$$

Summary of (a):

$$z^{[1]} = \begin{bmatrix} -1.5 \\ 3.5 \end{bmatrix}, \quad a^{[1]} = \begin{bmatrix} 0 \\ 3.5 \end{bmatrix}, \quad z^{[2]} = -3.5, \quad \hat{y} \approx 0.0293, \quad \mathcal{L} \approx 3.902.$$

1.2 (b) Gradient with respect to $a^{[1]}$

For BCE with a sigmoid output, for a single example:

$$\frac{\partial \mathcal{L}}{\partial z^{[2]}} = \hat{y} - y.$$

With $\hat{y} \approx 0.0293122$, $y = 1$,

$$\delta^{[2]} \equiv \frac{\partial \mathcal{L}}{\partial z^{[2]}} = 0.0293122 - 1 \approx -0.9706878.$$

The gradient w.r.t. the hidden activations is

$$\frac{\partial \mathcal{L}}{\partial a^{[1]}} = (W^{[2]})^\top \delta^{[2]} = \begin{bmatrix} 1.0 \\ -1.0 \end{bmatrix} (-0.9706878) = \begin{bmatrix} -0.9706878 \\ 0.9706878 \end{bmatrix}.$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial a^{[1]}} \approx \begin{bmatrix} -0.9707 \\ 0.9707 \end{bmatrix}}$$

1.3 (c) Gradients w.r.t. $W^{[1]}$ and x

ReLU backpropagation

ReLU derivative:

$$\text{ReLU}'(u) = \begin{cases} 1 & u > 0, \\ 0 & u \leq 0. \end{cases}$$

Here

$$z^{[1]} = \begin{bmatrix} -1.5 \\ 3.5 \end{bmatrix} \Rightarrow \text{ReLU}'(z^{[1]}) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Thus

$$\delta^{[1]} \equiv \frac{\partial \mathcal{L}}{\partial z^{[1]}} = \frac{\partial \mathcal{L}}{\partial a^{[1]}} \odot \text{ReLU}'(z^{[1]}) = \begin{bmatrix} -0.9706878 \\ 0.9706878 \end{bmatrix} \odot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.9706878 \end{bmatrix}.$$

Gradient w.r.t. $W^{[1]}$

Without regularization, for an affine layer $z^{[1]} = W^{[1]}x + b^{[1]}$:

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \delta^{[1]} x^\top.$$

Including L2 regularization:

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \delta^{[1]} x^\top + \lambda W^{[1]}.$$

Compute $\delta^{[1]} x^\top$:

$$x^\top = [2 \quad -1],$$

$$\delta^{[1]} x^\top = \begin{bmatrix} 0 \\ 0.9706878 \end{bmatrix} [2 \quad -1] = \begin{bmatrix} 0 \cdot 2 & 0 \cdot (-1) \\ 0.9706878 \cdot 2 & 0.9706878 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 1.9413756 & -0.9706878 \end{bmatrix}.$$

Regularization part:

$$\lambda W^{[1]} = 0.1 \begin{bmatrix} 0.5 & 1.0 \\ 0.5 & -2.0 \end{bmatrix} = \begin{bmatrix} 0.05 & 0.10 \\ 0.05 & -0.20 \end{bmatrix}.$$

Adding:

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \begin{bmatrix} 0 & 0 \\ 1.9413756 & -0.9706878 \end{bmatrix} + \begin{bmatrix} 0.05 & 0.10 \\ 0.05 & -0.20 \end{bmatrix} = \begin{bmatrix} 0.05 & 0.10 \\ 1.9913756 & -1.1706878 \end{bmatrix}.$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial W^{[1]}} \approx \begin{bmatrix} 0.0500 & 0.1000 \\ 1.9914 & -1.1707 \end{bmatrix}}$$

In particular, for the (1, 1) element:

$$\frac{\partial \mathcal{L}}{\partial W_{11}^{[1]}} = 0 \text{ (data term)} + 0.1 \cdot 0.5 = 0.05.$$

The gradient w.r.t. the first-layer bias is

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \delta^{[1]} = \begin{bmatrix} 0 \\ 0.9706878 \end{bmatrix},$$

with no regularization on $b^{[1]}$.

Gradient w.r.t. input x

For $z^{[1]} = W^{[1]}x + b^{[1]}$,

$$\frac{\partial \mathcal{L}}{\partial x} = (W^{[1]})^\top \delta^{[1]}.$$

We have

$$(W^{[1]})^\top = \begin{bmatrix} 0.5 & 0.5 \\ 1.0 & -2.0 \end{bmatrix}, \quad \delta^{[1]} = \begin{bmatrix} 0 \\ 0.9706878 \end{bmatrix}.$$

Therefore

$$\frac{\partial \mathcal{L}}{\partial x} = \begin{bmatrix} 0.5 & 0.5 \\ 1.0 & -2.0 \end{bmatrix} \begin{bmatrix} 0 \\ 0.9706878 \end{bmatrix} = \begin{bmatrix} 0.5 \cdot 0 + 0.5 \cdot 0.9706878 \\ 1.0 \cdot 0 + (-2.0) \cdot 0.9706878 \end{bmatrix} = \begin{bmatrix} 0.4853439 \\ -1.9413756 \end{bmatrix}.$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial x} \approx \begin{bmatrix} 0.4853 \\ -1.9414 \end{bmatrix}}$$

Regularization does not affect $\partial \mathcal{L} / \partial x$ because it depends only on the weights.

1.4 (d) Dying ReLU and weight updates

For a ReLU neuron j in the hidden layer,

$$a_j^{[1]} = \text{ReLU}(z_j^{[1]}) = \max(0, z_j^{[1]}),$$

$$\frac{\partial a_j^{[1]}}{\partial z_j^{[1]}} = \begin{cases} 1 & z_j^{[1]} > 0, \\ 0 & z_j^{[1]} \leq 0. \end{cases}$$

If neuron j is “dead”, then $z_j^{[1]} \leq 0$ for all inputs. Hence $\text{ReLU}'(z_j^{[1]}) = 0$, and

$$\delta_j^{[1]} = \frac{\partial \mathcal{L}}{\partial z_j^{[1]}} = \frac{\partial \mathcal{L}}{\partial a_j^{[1]}} \cdot \frac{\partial a_j^{[1]}}{\partial z_j^{[1]}} = \frac{\partial \mathcal{L}}{\partial a_j^{[1]}} \cdot 0 = 0.$$

Therefore, for the weights from the input to neuron j ,

$$\frac{\partial \mathcal{L}}{\partial W_{j,:}^{[1]}} = \delta_j^{[1]} x^\top + \lambda W_{j,:}^{[1]} = 0 \cdot x^\top + \lambda W_{j,:}^{[1]} = \lambda W_{j,:}^{[1]}.$$

The gradient with respect to its bias is

$$\frac{\partial \mathcal{L}}{\partial b_j^{[1]}} = \delta_j^{[1]} = 0,$$

since biases are not regularized here.

Implications.

- If $\lambda = 0$ (no regularization), then

$$\frac{\partial \mathcal{L}}{\partial W_{j,:}^{[1]}} = 0,$$

so gradient descent will not update $W_{j,:}^{[1]}$ or $b_j^{[1]}$ at all: the neuron is permanently dead.

- With L2 regularization ($\lambda > 0$), the update for the weights of neuron j becomes

$$W_{j,:}^{[1]} \leftarrow W_{j,:}^{[1]} - \eta \frac{\partial \mathcal{L}}{\partial W_{j,:}^{[1]}} = W_{j,:}^{[1]} - \eta \lambda W_{j,:}^{[1]} = (1 - \eta \lambda) W_{j,:}^{[1]}.$$

Thus the weights still change, but only due to regularization; they decay towards zero without receiving any learning signal from the data. The bias $b_j^{[1]}$ remains unchanged (its gradient is zero).

Answer. A dying ReLU neuron does not receive any data-driven gradient (its error term $\delta_j^{[1]}$ is zero). Without regularization, its weights and bias never change. With L2 regularization, its weights are updated only by the λW term, shrinking towards zero, while the neuron effectively stays inactive and does not “revive”. Therefore, from the perspective of learning useful features, the algorithm does not meaningfully update that neuron once it has died.

1.5 Network Diagram

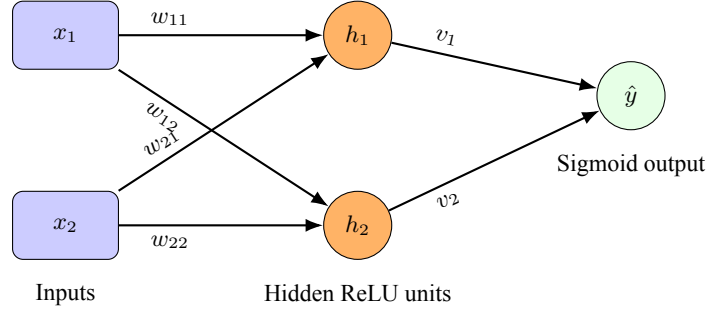


Figure 2: Architecture of the network: 2 inputs, 2 hidden ReLU neurons, and 1 sigmoid output neuron.

2 Question 2 (Problem Summary)

We study a deep MLP with 50 layers using the Sigmoid activation. We analyze:

- (a) Why gradients vanish in such a deep Sigmoid network.
- (b) What happens when we instead use ReLU and Batch Normalization, and whether this allows very large learning rates without divergence; we also discuss how ReLU can still fail and what BN does to mitigate failure.

2.1 (a) Vanishing Gradients in Deep Sigmoid Networks

The Sigmoid function is

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

For all z ,

$$0 \leq \sigma'(z) \leq 0.25.$$

When $z \gg 0$, $\sigma(z) \approx 1$ and $\sigma'(z) \approx 0$. When $z \ll 0$, $\sigma(z) \approx 0$ and $\sigma'(z) \approx 0$. The maximum derivative occurs at $z = 0$, where $\sigma(0) = \frac{1}{2}$ and

$$\sigma'(0) = \frac{1}{2} \left(1 - \frac{1}{2}\right) = \frac{1}{4}.$$

Consider backpropagation in a 50-layer network, with layers indexed $l = 1, \dots, 50$. For the loss $\mathcal{L}$ we have

$$\frac{\partial \mathcal{L}}{\partial z^{[1]}} = \left(\prod_{l=2}^{50} W^{[l]T} \text{diag}(\sigma'(z^{[l]})) \right) \frac{\partial \mathcal{L}}{\partial z^{[50]}}.$$

Taking norms, and assuming the weight matrices have moderate norms, the dominant factor is the product of activation derivatives:

$$\left\| \frac{\partial \mathcal{L}}{\partial z^{[1]}} \right\| \lesssim \left(\max_z |\sigma'(z)| \right)^{49} \left\| \frac{\partial \mathcal{L}}{\partial z^{[50]}} \right\| = (0.25)^{49} \left\| \frac{\partial \mathcal{L}}{\partial z^{[50]}} \right\|.$$

Since

$$(0.25)^{49} = 4^{-49} = 2^{-98} \approx 1.3 \times 10^{-30},$$

The value 0.25 is crucial because it is the *maximum* derivative of the Sigmoid. Since this maximum is already less than 1, repeated multiplication through many layers causes an exponential decay of gradients with depth.

-
- Diagram illustrating a deep neural network architecture for gradient computation. The network consists of an input layer with 49 nodes (blue circles), followed by Layer 1 (Sigmoid) with 3 nodes (purple circles), Hidden Layers 2-49 (Sigmoid) with 3 nodes each (yellow circles, highlighted with an orange box), Layer 50 (Sigmoid) with 3 nodes (purple circles), and an output layer with 1 node (red circle). The input is labeled x and the output is labeled y . A red arrow indicates the "Gradient from Loss" flowing from the output node back through the network. A purple arrow indicates the backpropagation of gradients from the hidden layers back to the input layer. The top left node of the input layer is labeled with the expression $\sigma'(z^{(l)})$ and the value $(1/4)^{49} \approx 10^{-30}$.

2.2 (b) ReLU, Large Learning Rates, and Batch Normalization

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$
$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

A student claims: “Because we use Batch Normalization, we can choose very large learning rates, e.g. in the range $[10, 20]$, and training will not diverge.”

(b1) Is the claim about very large learning rates correct?

- BN normalizes activations to have approximately zero mean and unit variance per mini-batch, making the optimization landscape smoother and allowing somewhat larger learning rates than without BN.
- However, the *parameters* can still change too abruptly if the learning rate is huge. Updates of size proportional to 10 or 20 are typically far too large and can cause the loss to oscillate or diverge.
- BN controls the scale of z within each batch, but it does not directly limit the magnitude of the weight updates.

7

(b2) Why can training still fail even with ReLU?

Compared to Sigmoid, ReLU helps with vanishing gradients because its derivative is 1 whenever $z > 0$. Nevertheless, ReLU networks can still suffer from other failures:

Dying ReLU. If a neuron's pre-activation becomes negative for all training examples, its output is always zero:

$$a^{[l]} = \text{ReLU}(z^{[l]}) = 0, \quad \text{ReLU}'(z^{[l]}) = 0.$$

Then the gradient through this neuron vanishes:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = 0,$$

and the neuron stops learning permanently ("dying ReLU"). Large or poorly chosen updates (e.g. due to too large a learning rate) can push many units into this inactive regime.

Exploding activations and gradients. If weights grow too large, the pre-activations $z^{[l]}$ and activations $a^{[l]}$ can become very large. This leads to very large gradients and unstable updates, causing training to diverge.

Internal covariate shift. As earlier layers change, the distribution of activations feeding into later layers keeps shifting. This can make optimization harder and increase the risk that some units become mostly inactive or that gradients become poorly scaled.

So even with ReLU, training can fail when the learning rate is too high or the network becomes poorly conditioned.

(b3) Role of Batch Normalization

Batch Normalization transforms pre-activations z_i into

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

(and typically then applies a learned affine transformation). This has several stabilizing effects:

- **Keeps inputs to ReLU in a controlled range.**

Since $\hat{z}_i$ has approximately zero mean and unit variance in each batch, roughly some fraction of values are positive and some negative. This reduces the chance that many ReLUs are always negative (dying) or always very large positive.

- **Reduces internal covariate shift.**

As parameters in earlier layers change, BN re-normalizes each batch, so the distribution of activations seen by later layers changes less dramatically. This makes optimization more stable.

- **Stabilizes gradient magnitudes.**

Controlling the variance of activations helps control the scale of backpropagated gradients, reducing exploding or vanishing gradients and making training less sensitive to initialization.

- **Allows moderately larger learning rates.**

Because activations and gradients are better scaled, one can often use a somewhat larger learning rate than in an unnormalized network, and training can converge faster.

However, BN does *not* guarantee that arbitrarily large learning rates (such as 10 or 20) will work. Extremely large learning rates can still cause large, unstable parameter updates that make training diverge, even when BN is used.

(2) ReLU + Batch Normalization Block

Conclusion. ReLU alleviates the classic vanishing gradient problem of Sigmoid, but training can still fail due to dying ReLUs, exploding activations, and poorly scaled updates. Batch Normalization stabilizes activations and gradients, mitigates these issues, and allows moderately larger learning rates, but it does not make training immune to divergence when the learning rate is chosen excessively large.

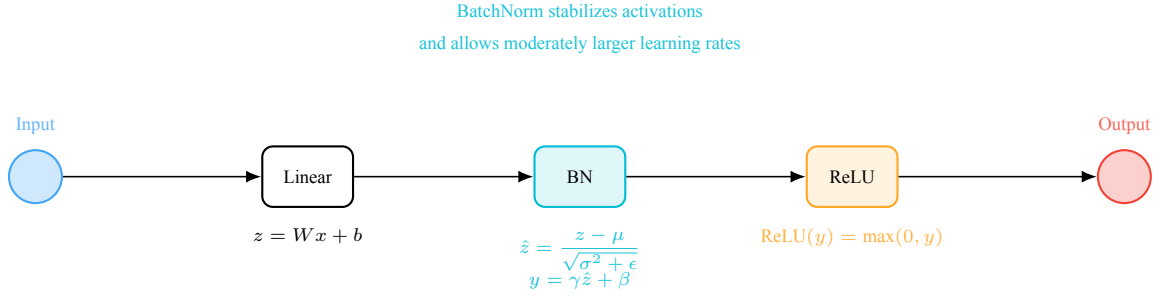


Figure 4: A ReLU + BatchNorm block: BN normalizes activations before ReLU, stabilizing training, but does not make arbitrarily large learning rates always safe.

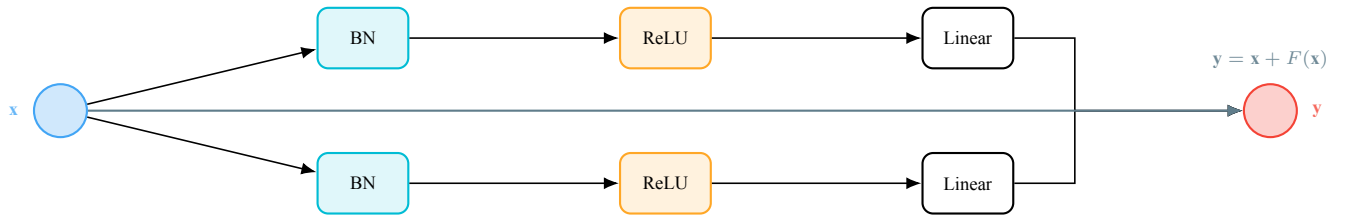


Figure 5: A simple residual block: skip connections help gradients flow backward through many layers, mitigating vanishing gradients.

3 Question 3: Vectorized Implementation of Backpropagation and Adam

3.1 Task 1: Computing Gradients for Layer 1

In this section, we implement the backward propagation for the first hidden layer (equipped with a ReLU activation function) using strictly vectorized operations in NumPy.

The mathematical formulations for the gradients of the pre-activation $dZ^{[1]}$, weights $dW^{[1]}$, and biases $db^{[1]}$ for a mini-batch of size m are as follows:

$$dZ^{[1]} = \left(W^{[2]T} dZ^{[2]} \right) \odot g'(Z^{[1]}) \quad (1)$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T \quad (2)$$

$$db^{[1]} = \frac{1}{m} \sum_{i=1}^m dZ^{[1](i)} \quad (3)$$

Where $\odot$ denotes the element-wise (Hadamard) product, and $g'(Z^{[1]})$ is the derivative of the ReLU function. The corresponding Python implementation is provided below:

```
# 1. Compute dZ1 (Gradient of ReLU pre-activation)
# (A1 > 0) creates a boolean mask that acts as the exact derivative of the ReLU
#    ↪ function.
dZ1 = np.dot(W2.T, dZ2) * (A1 > 0)

# 2. Compute dW1 and db1 explicitly
# The resulting dimensions will be (n_h, n_x) for dW1 and (n_h, 1) for db1.
dW1 = (1 / m) * np.dot(dZ1, X.T)

# The 'keepdims=True' argument is crucial to maintain the 2D shape (n_h, 1)
# instead of flattening the array into a rank-1 array (n_h,).
db1 = (1 / m) * np.sum(dZ1, axis=1, keepdims=True)
```

Listing 1: Vectorized Gradients Calculation for Layer 1

3.2 Task 2: Adam Optimizer Updates for $W^{[1]}$

In this task, we apply the Adam (Adaptive Moment Estimation) optimizer to update the weights of the first hidden layer ($W^{[1]}$). Adam combines the concepts of Momentum (tracking the gradient direction) and RMSProp (tracking the gradient magnitude) to adapt the learning rate individually for each parameter.

The mathematical formulation for the Adam update rule at timestep t is defined as follows:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) dW^{[1]} \quad (\text{Update First Moment}) \quad (4)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(dW^{[1]} \right)^{\odot 2} \quad (\text{Update Second Moment}) \quad (5)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (\text{Bias-Corrected First Moment}) \quad (6)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{Bias-Corrected Second Moment}) \quad (7)$$

$$W_{t+1}^{[1]} = W_t^{[1]} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \odot \hat{m}_t \quad (\text{Final Weight Update}) \quad (8)$$

Note that $(dW^{[1]})^{\odot 2}$ indicates the element-wise square of the gradient. The default hyperparameters are $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 0.01$ (learning rate), and $\epsilon = 10^{-8}$.

The corresponding vectorized Python implementation without any for loops is provided below:

```
# Hyperparameters (defined by standard default values and the problem description)
beta1 = 0.9
beta2 = 0.999
lr = 0.01
epsilon = 1e-8

# 1. Update First Moment (Momentum)
# Tracks the moving average of the gradient.
m_t_new = beta1 * m_t + (1 - beta1) * dW1

# 2. Update Second Moment (RMSProp)
# Tracks the moving average of the squared gradient (element-wise).
v_t_new = beta2 * v_t + (1 - beta2) * np.square(dW1)

# 3. Bias Correction
# Corrects the initialization bias towards zero during the early timesteps.
m_t_hat = m_t_new / (1 - beta1 ** t)
v_t_hat = v_t_new / (1 - beta2 ** t)

# 4. Final Weight Update
# Updates the weights using the adaptive learning rate.
W1_updated = W1 - lr * (m_t_hat / (np.sqrt(v_t_hat) + epsilon))
```

Listing 2: Adam Optimizer Update Rule for Layer 1 Weights

3.3 Task 3: Analytical Question on keepdims and Broadcasting

1. Why is keepdims=True required when computing $db^{[1]}$?

The gradient matrix $dZ^{[1]}$ has dimensions (n_h, m) . When we apply `np.sum(dZ1, axis=1)` to sum across the mini-batch examples (the columns), NumPy inherently collapses the dimension over which the sum is computed. Without setting `keepdims=True`, the output collapses into a 1D rank-1 array with shape $(n_h,)$, which is neither a row vector nor a column vector. By explicitly using `keepdims=True`, NumPy retains the collapsed dimension as a size of 1, resulting in a proper 2D column vector of shape $(n_h, 1)$. This strictly matches the required mathematical dimensions for the bias vector $b^{[1]}$.

2. What happens to Python Broadcasting in the next iteration if omitted?

If `keepdims` is omitted and $db^{[1]}$ (and subsequently the updated $b^{[1]}$) becomes a rank-1 array of shape $(n_h,)$, a critical alignment issue will occur during the forward propagation of the next training iteration. The equation for the linear transformation is $Z^{[1]} = W^{[1]}X + b^{[1]}$.

The matrix multiplication $W^{[1]}X$ yields a matrix of shape (n_h, m) . According to NumPy's broadcasting rules, which align trailing dimensions from right to left, an array of shape $(n_h,)$ is interpreted as a row vector $(1, n_h)$. NumPy will inappropriately attempt to broadcast this row vector across the (n_h, m) matrix.

- If $m \neq n_h$ (which is typical), the code will instantly crash with a `ValueError: operands could not be broadcast together`.
- If $m = n_h$ (an edge case), the program will not crash, but the biases will be added column-wise instead of row-wise, leading to a silent and catastrophic mathematical corruption that prevents the network from converging.

3.4 Complete Python Implementation

For clarity and grading purposes, the complete, vectorized implementation for calculating the gradients and applying the Adam optimizer updates without any for loops is provided below:

```
def backward_and_adam_update(X, A1, W2, dZ2, W1, b1, m, t, m_t, v_t):
    """
    Computes gradients for the first layer and updates W1 using the Adam Optimizer.
    """
    # --- Task 1: Compute Gradients for Layer 1 ---

    # 1. Compute dZ1 (Gradient of ReLU pre-activation)
    # (A1 > 0) creates a boolean mask that acts as the exact derivative of ReLU.
    dZ1 = np.dot(W2.T, dZ2) * (A1 > 0)

    # 2. Compute dW1 and db1 explicitly
    # Resulting shapes: dW1 -> (n_h, n_x), db1 -> (n_h, 1)
    dW1 = (1 / m) * np.dot(dZ1, X.T)
    db1 = (1 / m) * np.sum(dZ1, axis=1, keepdims=True)

    # --- Task 2: Adam Optimizer Updates for W1 ---

    # Hyperparameters (typical default values)
    beta1 = 0.9
    beta2 = 0.999
    lr = 0.01
    epsilon = 1e-8

    # 1. Update First Moment (Momentum)
    m_t_new = beta1 * m_t + (1 - beta1) * dW1

    # 2. Update Second Moment (RMSProp)
    v_t_new = beta2 * v_t + (1 - beta2) * np.square(dW1)

    # 3. Bias Correction for First and Second Moments
    m_t_hat = m_t_new / (1 - beta1 ** t)
    v_t_hat = v_t_new / (1 - beta2 ** t)

    # 4. Final Weight Update
    W1_updated = W1 - lr * (m_t_hat / (np.sqrt(v_t_hat) + epsilon))

    return W1_updated, m_t_new, v_t_new
```

Listing 3: Complete Vectorized Backward Propagation and Adam Update

4 Question 4: Theoretical Analysis of Dropout and Scaling

Setup: Consider a hidden neuron with a pre-activation value $a = 10$. The Dropout mechanism is applied with a drop probability $p = 0.5$, effectively acting as a Bernoulli mask during the training phase.

4.1 (a) Statistical Expectation in the Training Phase

During training, the neuron's output is a stochastic variable, y_{train} , which depends on the dropout state. The probability distribution is defined as:

$$y_{train} = \begin{cases} a & \text{with probability } (1 - p) = 0.5 \\ 0 & \text{with probability } p = 0.5 \end{cases}$$

To find the average behavior of this neuron during the forward pass, we calculate its expected value:

$$\mathbb{E}[y_{train}] = \sum y \cdot P(y) = (a \times 0.5) + (0 \times 0.5)$$

$$\mathbb{E}[y_{train}] = 10 \times 0.5 = 5$$

Result: The mean activation level that the subsequent layers learn to represent during training is $\boxed{5}$.

4.2 (b) Inference Discrepancy and Calibration Strategies

At test time (Inference), the stochastic Dropout is deactivated to ensure deterministic outputs. Thus, the neuron consistently yields $y_{test} = a = 10$.

The Mismatch: Comparing the two phases, we observe that $y_{test} = 10$ while $\mathbb{E}[y_{train}] = 5$. This factor-of-two difference ($y_{test} = \frac{1}{1-p} \mathbb{E}[y_{train}]$) leads to a magnitude shift in the input distribution of the following layers, which can significantly degrade model performance.

Mathematical Resolutions:

To maintain consistency between training and inference, we can employ one of the following two scaling techniques:

Option 1: Inverted Dropout Strategy (Active Learning)

This is the modern approach where we compensate for the dropped energy *during the training phase*. We scale the active neurons by a factor of $\frac{1}{1-p}$:

$$\hat{y}_{train} = \frac{y_{train}}{1-p} = \frac{y_{train}}{0.5} = 2 \cdot y_{train}$$

In this case, the new expectation becomes $\mathbb{E}[\hat{y}_{train}] = 10$, which aligns perfectly with the test-time output $y_{test} = 10$ without requiring any modifications during inference.

Option 2: Post-Training Weight Scaling (Classic Approach)

Alternatively, we can keep the training phase as is and apply a correction *during inference*. We scale the weights or the activation by the keep probability $(1-p)$:

$$\hat{y}_{test} = a \times (1-p) = 10 \times 0.5 = 5$$

By reducing the inference-time activation to 5, we match the expected signal level that the network was optimized for during the training process.

5 Question 5: Exoplanet Habitable Zone (Coding Challenge)

5.1 Phase 1: Proving the Deadlock of Linear Models

In this phase, we evaluate the performance of a simple linear classifier—specifically Logistic Regression—on the "Habitable Zone" dataset. The dataset follows a non-linear concentric circle structure, where habitable planets (Class 1) are located in an inner ring, surrounded by uninhabitable planets (Class 0).

Model Accuracy:

Upon training the Logistic Regression model on the training set and evaluating its performance on the test data, the achieved accuracy was approximately **41.67%**. This result is significantly lower than a standard threshold and reflects the model's inability to distinguish between the two classes.

Mathematical and Geometric Analysis of Failure:

The failure of this model is rooted in the concept of **Linear Separability**. A linear classifier (such as Logistic Regression or a single-layer Perceptron) is mathematically constrained to producing a straight decision boundary (a hyperplane) in the feature space.

As visualized in Figure 6, the classes in this dataset are distributed in a way that no single straight line can isolate the inner circle from the outer ring. Consequently, the linear boundary intersects both classes regardless of its orientation, leading to high classification error and proving that linear models are insufficient for capturing radial or non-linear patterns.

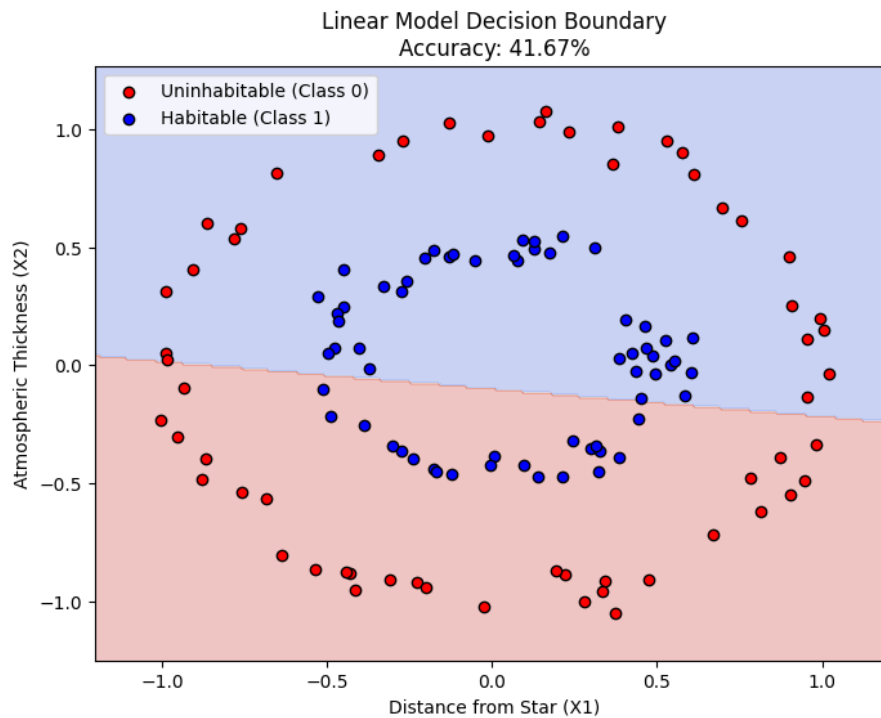


Figure 6: Decision boundary of the linear Logistic Regression model. The straight boundary illustrates the model's failure to adapt to the circular geometry of the data.

Python Implementation

The following code was implemented in Google Colab to generate the dataset, train the linear model, and visualize the resulting decision boundary:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 1. Generate the "Habitable Zone" dataset
X, y = make_circles(n_samples=600, noise=0.05, factor=0.5, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)

# 2. Train a Simple Linear Classifier (Logistic Regression without hidden layers)
linear_model = LogisticRegression()
linear_model.fit(X_train, y_train)

# 3. Calculate Accuracy on Test Data
y_pred = linear_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {accuracy * 100:.2f}%")

# 4. Plotting the Decision Boundary
def plot_decision_boundary(model, X, y, title):
    # Set min and max values and give it some padding
    x_min, x_max = X[:, 0].min() - 0.2, X[:, 0].max() + 0.2
    y_min, y_max = X[:, 1].min() - 0.2, X[:, 1].max() + 0.2
    h = 0.01 # Step size in the mesh

    # Generate a grid of points with distance h between them
```

```

xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

# Predict the function value for the whole grid
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot the contour and training examples
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
plt.scatter(X[y==0, 0], X[y==0, 1], color='red', label='Uninhabitable (Class 0)',
            ↪ edgecolors='k')
plt.scatter(X[y==1, 0], X[y==1, 1], color='blue', label='Habitable (Class 1)',
            ↪ edgecolors='k')

plt.xlabel('Distance from Star (X1)')
plt.ylabel('Atmospheric Thickness (X2)')
plt.title(title)
plt.legend()
plt.show()

plot_decision_boundary(linear_model, X_test, y_test, f"Linear Model Decision
            ↪ Boundary\nAccuracy: {accuracy * 100:.2f}%")

```

Listing 4: Logistic Regression Implementation for Phase 1

5.2 Phase 2: The Minimalist Architect Challenge

In this phase, we leverage the Universal Approximation Theorem to classify the concentric dataset using an MLP with a single hidden layer and ReLU activation functions. The objective is to find the absolute minimum number of hidden neurons required to achieve an accuracy exceeding 95% by successfully enclosing the inner class (habitable zone).

Geometric Interpretation of ReLU Neurons:

Answering the core geometric question: *What is the geometric equivalent of a ReLU neuron in the input space?* Each neuron in a hidden layer with a ReLU activation function effectively draws a straight line (a hyperplane) that divides the 2D input space into two half-spaces: one where the neuron is active (> 0) and one where it is inactive ($= 0$). To successfully isolate the inner circular class (Class 1) from the outer ring (Class 0), the network must intersect these half-spaces to form a **closed convex polygon** around the inner planets.

Based on the empirical decision boundaries shown in Figure 7, we can mathematically explain why certain architectures fail or succeed:

- **1 Neuron (Accuracy 41.67%):** Draws only a single line. It leaves the space completely open, acting just like a linear model, and therefore **never** solves the problem.
- **2 Neurons (Accuracy 49.17%):** Can draw two lines, creating an open region (such as a 'V' shape or a parallel strip). It is geometrically impossible to fully enclose a central region using only two lines, meaning it will **never** separate the classes.
- **3 Neurons (Accuracy 71.67%):** Three intersecting lines can theoretically form a closed triangle. However, as seen in our experiment, the optimization often settles into a local minimum (forming an open band) rather than a perfect triangle.
- **4 Neurons (Accuracy 100.00%):** Provides four lines, which easily form a closed quadrilateral (a diamond-like bounding box). The network successfully constructs a perfect trap around the habitable zone.

Conclusion: While 3 neurons is the theoretical absolute minimum to form a closed shape, **4 neurons** is the minimalist architecture required in practice to successfully and consistently separate this dataset with $> 95\%$ accuracy.

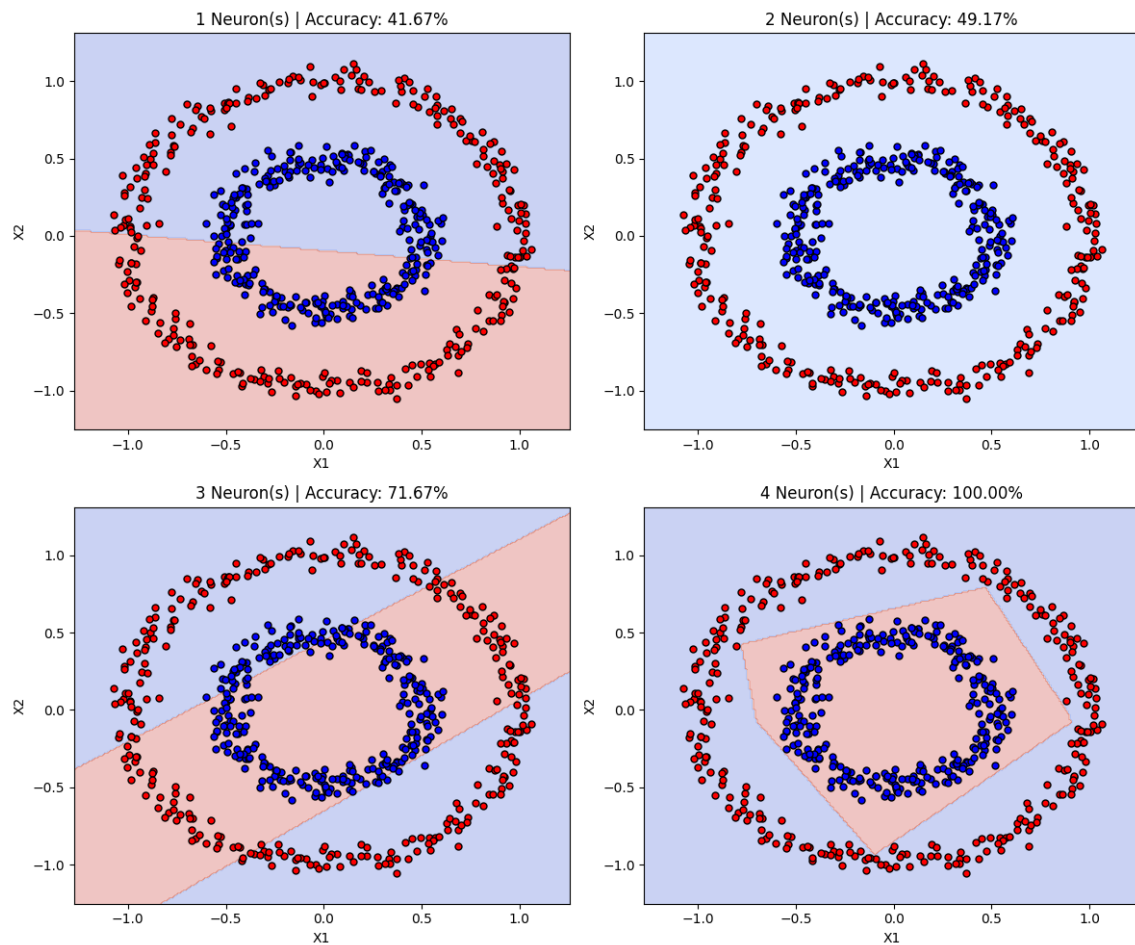


Figure 7: Geometric analysis of the decision boundary formed by an MLP with a single hidden layer containing 1, 2, 3, and 4 ReLU neurons. A closed polygon (achieved here with 4 neurons) is necessary to enclose the inner class.

Python Implementation

The following Colab script demonstrates the iterative training of the minimal MLP models using the lbfgs solver, which is highly optimized for small datasets:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# 1. Generate the dataset
X, y = make_circles(n_samples=600, noise=0.05, factor=0.5, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)

# List of neurons to test
neurons_list = [1, 2, 3, 4]

plt.figure(figsize=(12, 10))

for i, n in enumerate(neurons_list):
    # 2. Build and train MLP with 'n' neurons in the single hidden layer
    # We use 'lbfgs' solver which is highly efficient for small datasets/architectures
```

```

mlp = MLPClassifier(hidden_layer_sizes=(n,), activation='relu', solver='lbfgs',
    ↪ max_iter=2000, random_state=42)
mlp.fit(X_train, y_train)

# 3. Predict and calculate accuracy
y_pred = mlp.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Number of Neurons: {n} -> Test Accuracy: {accuracy * 100:.2f}%")

# 4. Plot Decision Boundary
plt.subplot(2, 2, i + 1)
x_min, x_max = X[:, 0].min() - 0.2, X[:, 0].max() + 0.2
y_min, y_max = X[:, 1].min() - 0.2, X[:, 1].max() + 0.2
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01), np.arange(y_min, y_max, 0.01))

Z = mlp.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
plt.scatter(X[y==0, 0], X[y==0, 1], color='red', label='Class 0', edgecolors='k', s
    ↪ =25)
plt.scatter(X[y==1, 0], X[y==1, 1], color='blue', label='Class 1', edgecolors='k',
    ↪ s=25)

plt.title(f"{n} Neuron(s) | Accuracy: {accuracy * 100:.2f}%")
plt.xlabel('X1')
plt.ylabel('X2')

plt.tight_layout()
plt.show()

```

Listing 5: MLP Minimum Architecture Search

5.3 Phase 3: The Activation Trap (Vanishing Gradients)

In this final phase, we investigate the impact of activation functions on the convergence speed of a relatively deep architecture (3 hidden layers, 10 neurons each) trained with a learning rate of 0.01.

Loss Curve Observation:

As illustrated in Figure 8, there is a stark contrast in the learning dynamics of the two networks. The loss curve for the ReLU-activated network (blue line) decreases smoothly and rapidly towards zero, indicating successful and robust convergence. Conversely, the loss curve for the Sigmoid-activated network (red line) remains entirely flat and stagnant from the very first epochs, demonstrating a complete failure to optimize the weights.

Identification of the Phenomenon:

This catastrophic failure in the Sigmoid-based network is a textbook manifestation of the **Vanishing Gradient Problem**. As the network depth increases, the early hidden layers fail to receive meaningful gradient updates, effectively trapping the network in its initial state and halting the learning process.

Mathematical Justification via Derivatives:

The root cause of this phenomenon lies in the mathematical properties of the activation functions' derivatives applied during backpropagation via the Chain Rule:

- **Sigmoid Activation:** The derivative of the sigmoid function, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, has an absolute maximum value of 0.25 (at $z = 0$). During backpropagation through 3 hidden layers, the error gradient from the output layer is continuously multiplied by these fractional derivatives. The compound effect is at best $(0.25)^3 \approx 0.015$, and practically much closer to zero. This exponential decay causes the gradients to "vanish" before they can update the weights of the initial layers.
- **ReLU Activation:** The derivative of the Rectified Linear Unit is exactly 1 for all positive inputs ($z > 0$). This constant gradient ensures that active neurons backpropagate the error signal without any attenuation (repeated multiplication by 1). Consequently, early layers receive strong, unaltered gradients, allowing the deep network to update its weights and converge rapidly.

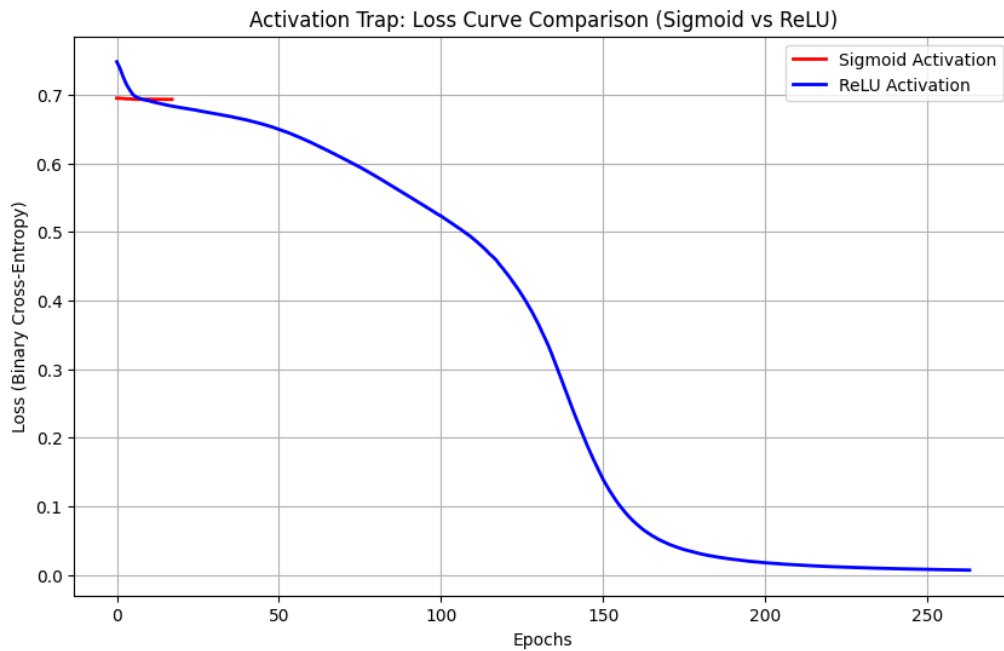


Figure 8: Loss curve comparison between Sigmoid and ReLU activations. The flat red line practically demonstrates the Vanishing Gradient problem in deep Sigmoid networks, whereas the ReLU network converges successfully.

Python Implementation

The execution and plotting of the loss curves were implemented as follows:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

# 1. Generate the dataset
X, y = make_circles(n_samples=600, noise=0.05, factor=0.5, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)

# 2. Common hyperparameters (3 hidden layers with 10 neurons each, LR = 0.01)
common_params = {
    'hidden_layer_sizes': (10, 10, 10),
    'learning_rate_init': 0.01,
    'solver': 'sgd', # SGD perfectly illustrates the Vanishing Gradient problem
    'max_iter': 1000,
    'random_state': 42
}

# 3. Train Model 1: Sigmoid (Logistic)
mlp_sigmoid = MLPClassifier(activation='logistic', **common_params)
mlp_sigmoid.fit(X_train, y_train)

# 4. Train Model 2: ReLU
mlp_relu = MLPClassifier(activation='relu', **common_params)
mlp_relu.fit(X_train, y_train)

# 5. Plot the Loss Curves
plt.figure(figsize=(10, 6))
plt.plot(mlp_sigmoid.loss_curve_, label='Sigmoid Activation', color='red',
    ↪ linewidth=2)
plt.plot(mlp_relu.loss_curve_, label='ReLU Activation', color='blue', linewidth=2)
```

```
plt.title('Activation Trap: Loss Curve Comparison (Sigmoid vs ReLU)')
plt.xlabel('Epochs')
plt.ylabel('Loss (Binary Cross-Entropy)')
plt.legend()
plt.grid(True)
plt.show()

# Print accuracies for reference
print(f"Sigmoid Test Accuracy: {mlp_sigmoid.score(X_test, y_test)*100:.2f}%")
print(f"ReLU Test Accuracy: {mlp_relu.score(X_test, y_test)*100:.2f}%")
```

Listing 6: Comparing Convergence: Sigmoid vs ReLU